Now, let's start our Redis server:

```
$ docker-compose up -d redis
```

We can see Redis starting up by viewing the logs:

```
$ docker-compose logs redis
Attaching to myapp_redis_1
redis_1  | 1:C 15 Jan 2019 10:03:52.794 # oO0OoO0OoO0Oo Redis is starting oO
0OoO0OoO0Oo
redis_1  | 1:C 15 Jan 2019 10:03:52.794 # Redis version=5.0.3, bits=64,
commit=00000000, modified=0, pid=1, just started
«...»
redis_1  | 1:M 15 Jan 2019 10:03:52.796 * Running mode=standalone, port=6379
«...»
redis_1  | 1:M 15 Jan 2019 10:03:52.796 # Server initialized
«...»
redis_1  | 1:M 15 Jan 2019 10:03:52.796 * Ready to accept connections
```

Great! We've successfully set up Redis as a new service for our application.

Manually Connecting to the Redis Server

We've just started Redis using Compose and saw from the output that it was running. However, since we're still getting familiar with Docker, let's manually connect to the Redis server and interact with it to prove to ourselves it really does work.

A quick way to do this is using the Redis command-line interface (redis-cli). We can leverage the same redis image, which already has redis-cli installed. Handy.

Rather than having to set up a new, separate service in Compose, we can piggyback on the existing redis service, since it uses the redis image we need. Using what we learned in Running One-Off Commands, on page 54, we can run redis-cli and connect to our Redis server with the following command:

```
$ docker-compose run --rm redis redis-cli -h redis
```

This command says, "In a throwaway container (--rm) for the redis service, run the command redis-cli -h redis." On running it, you should see the standard Redis prompt showing the hostname and port it's running on:

```
redis:6379>
```

Feel free to play around. For example, try running the ping command, which should give you the "PONG" response. When you're done, exit with the command quit—this will terminate the Redis client and, as a result, the container.